

# Análisis de Malware en Android con NATICUSdroid

Claudia Maria Rocha Hernandez, Sebastian Bedoya Restrepo, David Guillermo Naranjo Ochoa

**Resumen**—Este trabajo presenta la fase inicial de un sistema de detección de malware en aplicaciones Android utilizando el dataset NATICUSdroid. Debido al aumento de aplicaciones maliciosas que explotan el sistema de permisos, se plantea un enfoque de aprendizaje supervisado formulado como un problema de clasificación binaria. Se analiza la estructura del dataset, compuesto por más de 29000 aplicaciones y 86 características basadas en permisos, con el fin de comprender su comportamiento y su utilidad para el modelado. Adicionalmente, se revisan trabajos relacionados que evidencian la efectividad de modelos de ensamble y técnicas de selección de características en este tipo de problemas. Estos elementos permiten establecer una base clara para el desarrollo de modelos de detección en etapas posteriores.

**Index Terms**—Android, detección de malware, aprendizaje automático, clasificación binaria, selección de características

## I. INTRODUCCIÓN

ANDROID, desarrollado por Google, es actualmente el sistema operativo móvil más utilizado a nivel mundial, con miles de millones de dispositivos activos y un amplio ecosistema de aplicaciones distribuidas tanto en tiendas oficiales como en repositorios de terceros. Sin embargo, esta masificación también lo convierte en un blanco atractivo para el desarrollo de software malicioso. Distintos reportes han evidenciado incidentes en los que aplicaciones aparentemente legítimas explotan vulnerabilidades del sistema o recolectan información sensible.

En el contexto de la seguridad informática, la revisión manual del código ha sido una estrategia tradicional, aunque costosa y limitada. Por esta razón, han surgido enfoques basados en el análisis del comportamiento, especialmente a través de los permisos solicitados por las aplicaciones. Estos permiten inferir accesos a recursos sensibles y detectar patrones anómalos. Investigaciones previas han demostrado una relación significativa entre ciertos conjuntos de permisos y la presencia de malware en Android [3].

El dataset NATICUSdroid recopila información de más de 29000 aplicaciones clasificadas como benignas o maliciosas. Esto permite aplicar técnicas de aprendizaje automático para abordar el problema como una tarea de clasificación binaria.

El desarrollo de estos sistemas resulta relevante para fortalecer la seguridad en Android, mejorar la detección temprana de amenazas y comprender la evolución del malware.

Claudia Maria Rocha Hernandez: claudia.rochah@udea.edu.co  
Sebastian Bedoya Restrepo: sebastian.bedoya7@udea.edu.co  
David Guillermo Naranjo Ochoa: guillermo.naranjo@udea.edu.co

## II. DESCRIPCIÓN DEL PROBLEMA

### II-A. Contexto del problema

En el contexto actual de la seguridad en dispositivos móviles, la detección de aplicaciones maliciosas en Android se ha convertido en un desafío relevante debido al crecimiento constante de amenazas cada vez más sofisticadas. Los enfoques tradicionales basados en firmas, comúnmente utilizados por soluciones antivirus, han perdido efectividad frente a malware moderno que emplea técnicas como la ofuscación y la mutación de código para evadir mecanismos de detección estáticos.

Ante esta limitación, el uso de técnicas de Machine Learning surge como una alternativa adecuada, ya que permite analizar grandes volúmenes de datos y descubrir patrones complejos que no son evidentes mediante enfoques tradicionales. En particular, el análisis de los permisos solicitados por las aplicaciones representa una fuente de información clave, dado que estos reflejan el nivel de acceso al sistema y pueden evidenciar comportamientos potencialmente maliciosos. Estudios previos han demostrado que existe una relación significativa entre ciertos conjuntos de permisos y la presencia de malware en Android [3].

En este contexto, el presente trabajo aborda el problema de la detección automática de malware mediante técnicas de aprendizaje automático, partiendo de un análisis exploratorio del dataset y culminando en la definición de un modelo de clasificación capaz de discriminar entre aplicaciones benignas y maliciosas.

### II-B. Composición de la base de datos

El dataset utilizado corresponde a NATICUSdroid (Android Permission Dataset), el cual se encuentra disponible en el repositorio UCI Machine Learning. Este conjunto de datos está compuesto por 29333 instancias y 86 variables, donde cada instancia representa una aplicación Android clasificada como benigna o maliciosa, definiendo así un problema de clasificación binaria.

Las variables del dataset corresponden a permisos solicitados por las aplicaciones. Cada característica representa un permiso específico del sistema Android, y su valor indica su presencia (1) o ausencia (0) en la aplicación. Esta representación permite modelar el comportamiento de las aplicaciones mediante un enfoque de análisis estático basado en permisos, ampliamente utilizado en el ámbito de la ciberseguridad.

En cuanto a la codificación, las variables independientes son de tipo binario, mientras que la variable objetivo, denominada *Result*, es también binaria y representa la clase de la aplicación (0 para aplicaciones benignas y 1 para aplicaciones maliciosas). Este tipo de estructura

resulta especialmente adecuado para modelos de aprendizaje supervisado, ya que no requiere transformaciones adicionales y facilita la aplicación de algoritmos clásicos como Random Forest, Máquinas de Vectores de Soporte y Regresión Logística.

Un aspecto relevante del dataset es la ausencia de valores faltantes, lo que elimina la necesidad de aplicar técnicas de imputación y simplifica el preprocesamiento de los datos. Adicionalmente, al tratarse de un conjunto de datos tabular con variables discretas binarias, no es necesario realizar procesos de normalización, lo cual favorece el uso de modelos basados en árboles. No obstante, es posible la existencia de redundancia entre permisos, lo que sugiere la aplicación de técnicas de selección de características en etapas posteriores.

En conjunto, estas características hacen que el dataset NATICUSDroid sea adecuado para tareas de clasificación en ciberseguridad, al proporcionar una representación estructurada y consistente del comportamiento de las aplicaciones.

### II-C. Paradigma de aprendizaje y justificación

El enfoque seleccionado para abordar el problema es el aprendizaje supervisado, formulado como una tarea de clasificación binaria. Esta decisión se fundamenta en la estructura del dataset, donde cada instancia cuenta con una etiqueta de salida que indica si la aplicación es benigna o maliciosa. La disponibilidad de datos etiquetados permite entrenar modelos capaces de aprender una función de mapeo entre los patrones de permisos y la clase correspondiente, facilitando la predicción sobre nuevas aplicaciones no observadas previamente.

Desde una perspectiva técnica, esta formulación es coherente con la naturaleza del problema, ya que se busca discriminar entre dos estados mutuamente excluyentes. El espacio de características, compuesto por variables binarias, permite capturar relaciones complejas entre permisos, lo que resulta adecuado para modelos capaces de manejar alta dimensionalidad. Asimismo, este enfoque permite evaluar el desempeño mediante métricas relevantes en seguridad informática, como la tasa de falsos positivos y la medida F, las cuales son fundamentales para minimizar errores críticos en la detección.

Adicionalmente, la selección de este paradigma es consistente con lo reportado en la literatura reciente. Estudios como el de Mathur et al. [2] han demostrado el alto desempeño de modelos supervisados en la detección de malware en Android, mientras que propuestas como la de Feng y Zhang [1] evidencian la importancia de integrar técnicas de selección de características para optimizar el rendimiento en entornos dinámicos. Estos enfoques serán abordados con mayor detalle en la siguiente sección.

## III. ESTADO DEL ARTE

El estudio de la seguridad en el ecosistema Android ha evolucionado para enfrentar amenazas que explotan el sistema de permisos con el fin de comprometer la

privacidad del usuario. En este contexto, Mathur et al. [2] proponen el framework NATICUSDroid, cuyo objetivo principal es superar las limitaciones de estudios previos basados en conjuntos de datos obsoletos. Su enfoque se centra en la relevancia de los permisos tanto nativos del sistema operativo como personalizados. Para ello, los autores realizan un análisis estadístico sobre una base de datos de más de 29000 aplicaciones recolectadas entre 2010 y 2019, aplicando técnicas de selección de características y evaluando múltiples algoritmos de aprendizaje automático. Los resultados obtenidos indican que el modelo basado en Random Forest presenta el mejor desempeño, alcanzando una precisión cercana al 97%, lo que demuestra que la combinación de permisos actuales constituye un predictor robusto para la detección de malware moderno.

Por otro lado, la necesidad de gestionar de forma eficiente grandes volúmenes de características, como los presentes en el dataset NATICUSDroid, es abordada en el trabajo de Feng y Zhang [1]. Estos autores analizan el problema desde una perspectiva de optimización en la selección de características dentro de entornos dinámicos. Su propuesta introduce un método de selección incremental basado en un vector de regularización, el cual permite actualizar la información sin necesidad de reentrenar completamente los modelos. Este enfoque resulta especialmente relevante en escenarios donde los datos evolucionan constantemente, como ocurre con la aparición continua de nuevas aplicaciones y permisos en Android. Los resultados experimentales muestran que este método supera a técnicas convencionales, lo que sugiere su potencial integración en sistemas de detección de malware para mejorar su eficiencia y capacidad de adaptación en entornos reales.

Un tercer estudio relevante es el de Liu et al. [4], quienes presentan una comparación sistemática entre modelos tradicionales de aprendizaje automático y enfoques basados en aprendizaje profundo. Su principal aporte es mostrar que, en varios conjuntos de datos recientes, modelos más simples como Random Forest o CatBoost pueden alcanzar un desempeño comparable, e incluso superior, al de arquitecturas profundas más costosas. El trabajo enfatiza la necesidad de realizar benchmarking rigurosos para evitar conclusiones sesgadas y resalta que, en este dominio, la eficiencia computacional también debe considerarse al evaluar soluciones.

Finalmente, Alhogail y Alharbi [5] proponen un esquema en dos etapas para detección y categorización de malware en Android. Primero, aplican una clasificación binaria para identificar aplicaciones maliciosas; luego, usan una segunda etapa multicategoría para distinguir entre familias como adware, banking Trojan, SMS malware y riskware. En su metodología combinan selección de características mediante chi-cuadrado y Select-From-Model con clasificadores clásicos, donde Random Forest destaca por su equilibrio entre precisión e interpretabilidad. Este estudio es especialmente útil para nuestro trabajo porque refuerza el valor de la ingeniería de características y de los modelos de ensamble en problemas de seguridad móvil.

En conjunto, estos trabajos muestran que la detección

de malware en Android puede abordarse desde enfoques estáticos y dinámicos, con o sin aprendizaje profundo, pero siempre con una fuerte dependencia de la calidad de las características y de la metodología de validación. Para este proyecto, esas ideas justifican el uso de un modelo supervisado con selección de características y una evaluación cuidadosa de su capacidad de generalización.

#### IV. ENTRENAMIENTO Y EVALUACIÓN DE LOS MODELOS

##### IV-A. Configuración experimental

Todos los modelos se entrenaron y evaluaron sobre la misma partición estratificada del dataset, con 70 % de las muestras para entrenamiento y 30 % para prueba, usando `random_state = 42`. Para garantizar una comparación justa, los notebooks de regresión logística, SVM, KNN, red neuronal y Random Forest se unificaron para usar la misma rutina de preparación de datos del proyecto, de modo que todos parten exactamente del mismo conjunto de entrenamiento y del mismo conjunto de prueba.

En los modelos sensibles a la escala de las variables, como regresión logística, SVM y KNN, se aplicó *StandardScaler* antes del ajuste. En cambio, Random Forest y la red neuronal se entrenaron directamente con la representación binaria de permisos. La selección de hiperparámetros se llevó a cabo mediante validación cruzada estratificada, optimizando principalmente el *F1-score*, ya que en detección de malware es especialmente importante mantener un equilibrio adecuado entre falsos positivos y falsos negativos.

Cuadro I: Protocolo común de evaluación de los modelos.

| Elemento                | Configuración adoptada                                                 |
|-------------------------|------------------------------------------------------------------------|
| Partición principal     | 70 % entrenamiento y 30 % prueba, con muestreo estratificado           |
| Semilla                 | <code>random_state = 42</code>                                         |
| Variable objetivo       | <code>Result</code> (0: benigna, 1: maliciosa)                         |
| Preprocesamiento        | Escalado estándar en regresión logística, SVM y KNN                    |
| Criterio de ajuste      | Maximización del <i>F1-score</i> en validación                         |
| Métricas comunes        | Accuracy, precision, recall, <i>F1-score</i> , especificidad y ROC-AUC |
| Intervalos de confianza | Bootstrap al 95 % para accuracy y <i>F1-score</i>                      |
| Conjuntos reportados    | Entrenamiento, validación y prueba                                     |

Las métricas elegidas responden directamente a la naturaleza del problema. La *accuracy* resume el porcentaje global de clasificaciones correctas; la *precision* refleja qué tan confiables son las alertas de malware; el *recall* indica la capacidad para detectar aplicaciones maliciosas reales; el *F1-score* sintetiza el equilibrio entre *precision* y *recall*; la especificidad mide el reconocimiento correcto de aplicaciones benignas; y el ROC-AUC resume la capacidad discriminativa del modelo a través de diferentes umbrales. En conjunto, estas métricas permiten valorar no solo el desempeño global, sino también la estabilidad y la capacidad real de generalización de cada enfoque.

El Cuadro II resume la malla de hiperparámetros o la configuración de entrenamiento utilizada para cada modelo. Este cuadro sirve como referencia para interpretar los

resultados presentados posteriormente en la subsección 4.2.

Cuadro II: Hiperparámetros y configuraciones evaluadas por modelo.

| Modelo              | Configuración evaluada                                                                                                                                                                             |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Regresión logística | C: 0.01, 0.1, 1, 10; solver: lbfgs, liblinear; max_iter=5000; validación estratificada de 5 pliegues.                                                                                              |
| SVM lineal          | C: 0.01, 0.1, 1, 10, 100; loss=squared_hinge; dual=auto; max_iter=4000 en validación y 5000 en el ajuste final; validación estratificada de 5 pliegues.                                            |
| KNN                 | n_neighbors: 3, 5, 7, 9, 11; weights: uniform, distance; metric: euclidean, manhattan; validación estratificada de 5 pliegues.                                                                     |
| Red neuronal        | Arquitectura 256→128→64→2; activaciones ReLU; dropout 0.3 y 0.2; Adam con tasa de aprendizaje 0.001; weight decay $10^{-5}$ ; lote de 64; 50 épocas.                                               |
| Random Forest       | n_estimators: 200, 400; max_depth: None, 25; min_samples_split: 2, 5; min_samples_leaf: 1, 2; max_features: sqrt; class_weight: None, balanced_subsample; validación estratificada de 10 pliegues. |

##### IV-B. Resultados del entrenamiento de los modelos

**IV-B1. Regresión logística:** La regresión logística se utilizó como línea base paramétrica. La mejor configuración fue **C = 0.1** con **solver = lbfgs**. Durante la validación cruzada estratificada de 5 pliegues alcanzó un *F1-score* promedio de 0.9603. En entrenamiento registró *accuracy* de 0.9610, *F1-score* de 0.9613 y *ROC-AUC* de 0.9897, mientras que en prueba llegó a *accuracy* de 0.9590, *F1-score* de 0.9593 y *ROC-AUC* de 0.9888. La brecha entre entrenamiento y prueba fue mínima, lo que sugiere un buen nivel de generalización y poco sobreajuste.

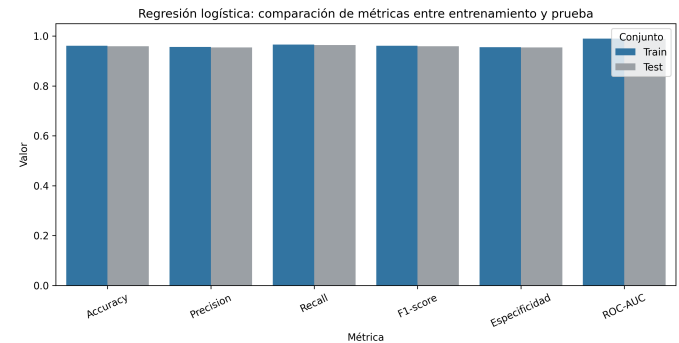


Figura 1: Comparación entre métricas de entrenamiento y prueba para la regresión logística.

La Figura 1 confirma que la regresión logística mantiene un comportamiento muy similar en entrenamiento y en prueba. Esto es consistente con la baja complejidad relativa del modelo y con la ausencia de una brecha pronunciada de generalización.

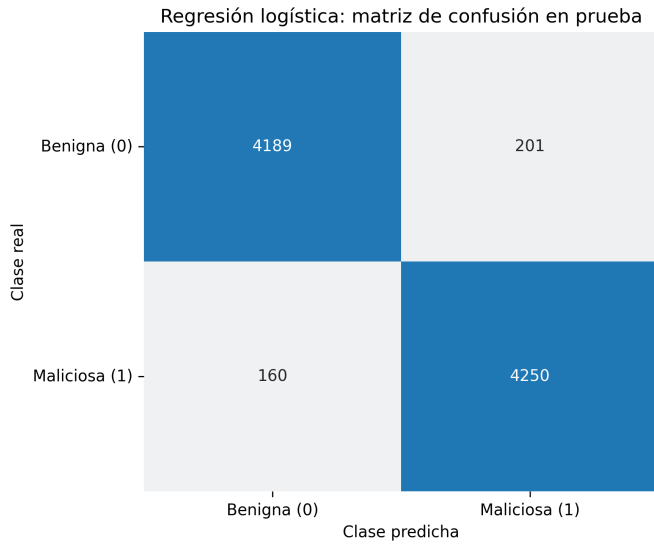


Figura 2: Matriz de confusión de la regresión logística sobre el conjunto de prueba.

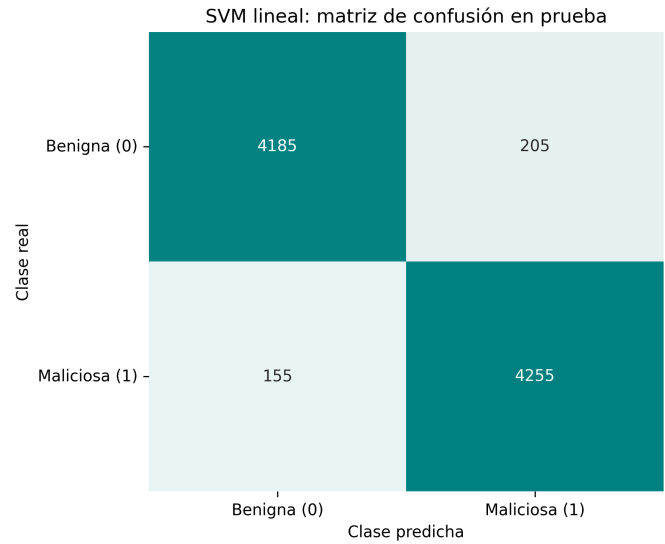


Figura 4: Matriz de confusión de la SVM lineal sobre el conjunto de prueba.

*IV-B2. Máquina de vectores de soporte:* La SVM lineal seleccionó como mejor valor de regularización  $C = 0.01$ . En validación cruzada estratificada de 5 pliegues alcanzó un  $F1$ -score promedio de 0.9607. En entrenamiento registró  $accuracy$  de 0.9612,  $F1$ -score de 0.9615 y  $ROC$ -AUC de 0.9893, mientras que en prueba llegó a  $accuracy$  de 0.9591,  $F1$ -score de 0.9594 y  $ROC$ -AUC de 0.9885. Al igual que en la regresión logística, las diferencias entre ambos conjuntos fueron reducidas, lo que confirma un comportamiento estable.

*IV-B3. KNN:* KNN se empleó como alternativa no paramétrica basada en proximidad. La mejor configuración fue  $n\_neighbors = 7$ ,  $weights = distance$  y  $metric = manhattan$ . En validación cruzada estratificada de 5 pliegues alcanzó un  $F1$ -score promedio de 0.9645. En entrenamiento registró  $accuracy$  de 0.9809 y  $F1$ -score de 0.9808, mientras que en prueba obtuvo  $accuracy$  de 0.9644 y  $F1$ -score de 0.9645. Aunque el rendimiento absoluto fue competitivo, la brecha entre entrenamiento y prueba fue mayor que en los modelos lineales, lo que sugiere una sensibilidad más alta al sobreajuste.

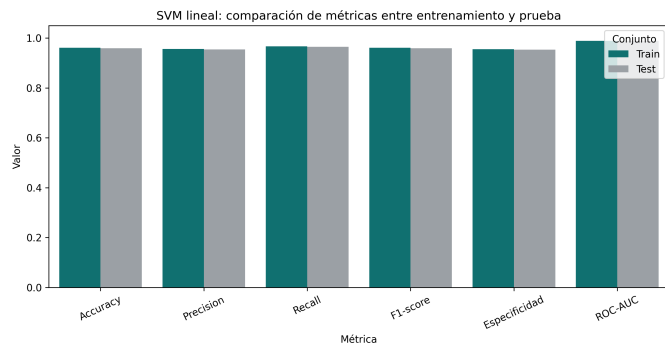


Figura 3: Comparación entre métricas de entrenamiento y prueba para la SVM lineal.

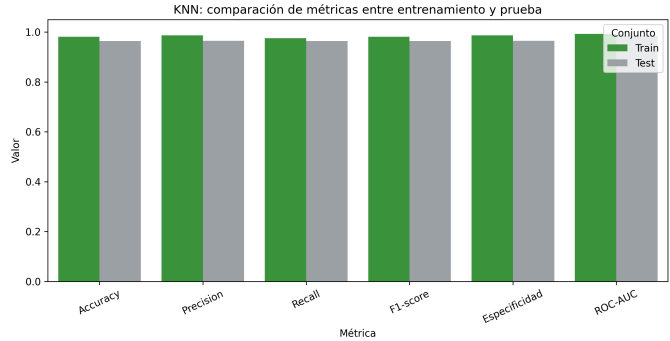


Figura 5: Comparación entre métricas de entrenamiento y prueba para KNN.

La Figura 3 muestra que la SVM mantiene una relación muy equilibrada entre entrenamiento y prueba. Esto respalda su uso como modelo lineal competitivo y estable en este problema de detección.

La Figura 5 permite observar una separación más clara entre el rendimiento en entrenamiento y en prueba. Esto indica que KNN ajusta muy bien los datos vistos, pero pierde parte de ese desempeño al generalizar a muestras no observadas.

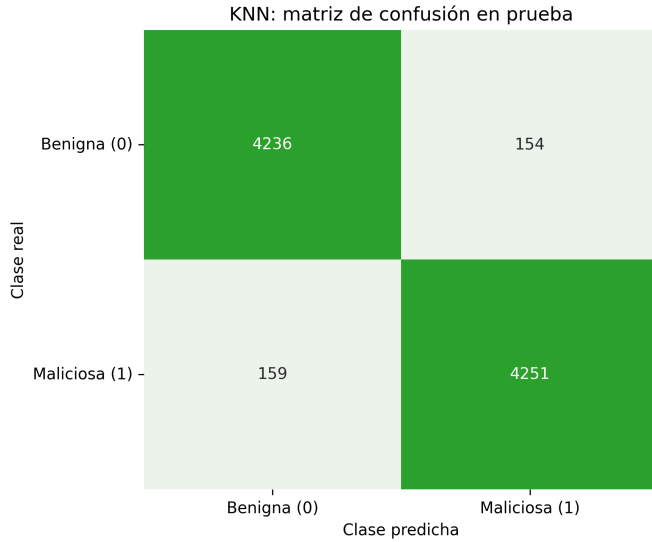


Figura 6: Matriz de confusión del modelo KNN sobre el conjunto de prueba.

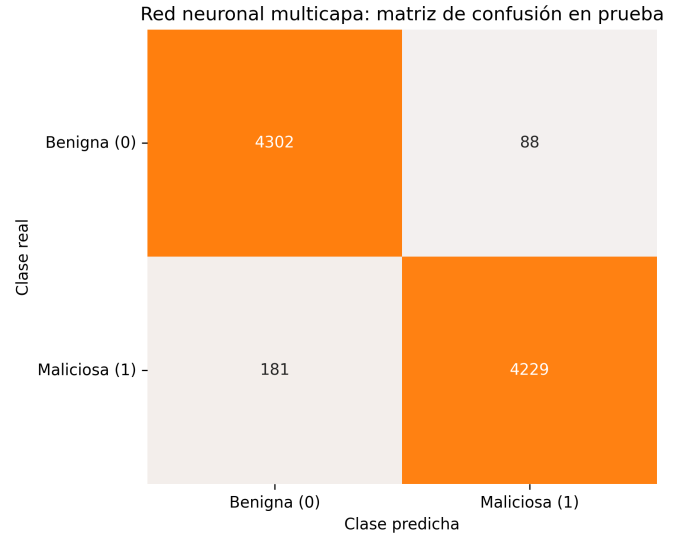


Figura 8: Matriz de confusión de la red neuronal multicapa sobre el conjunto de prueba.

*IV-B4. Red neuronal multicapa:* La red neuronal multicapa se implementó en PyTorch con tres capas ocultas densas, funciones ReLU, *dropout* y optimización con Adam. En este caso no se realizó una malla exhaustiva, sino que se trabajó con una configuración fija. En entrenamiento alcanzó *accuracy* de 0.9806, *F1-score* de 0.9805 y *ROC-AUC* de 0.9975, mientras que en prueba registró *accuracy* de 0.9694, *F1-score* de 0.9692 y *ROC-AUC* de 0.9924. Este comportamiento la ubica entre los modelos con mejor desempeño global, aunque con una brecha de generalización moderada frente al conjunto de entrenamiento.

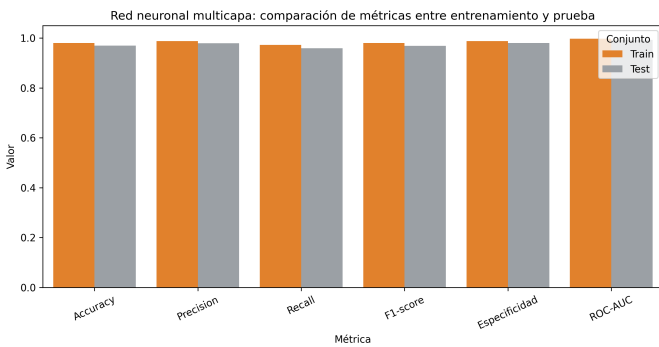


Figura 7: Comparación entre métricas de entrenamiento y prueba para la red neuronal multicapa.

La Figura 7 evidencia una mejora clara frente a los modelos lineales en las métricas globales, aunque también muestra una diferencia visible entre entrenamiento y prueba. Esa brecha sigue siendo aceptable y sugiere una buena capacidad de discriminación sin un sobreajuste extremo.

*IV-B5. Random Forest:* Random Forest fue el modelo de ensamble de árboles que tomamos como referencia principal y, además, el enfoque más cercano a la estrategia que decidimos retomar del trabajo de Feng y Zhang [1]. Aunque ese artículo se centra en selección incremental de características mediante un vector de regularización y una idea de *replay*, su aporte más valioso para nuestro proyecto fue recordar que, en problemas dinámicos y de alta dimensionalidad, no basta con clasificar bien: también conviene priorizar las variables más informativas y conservar estabilidad cuando el entorno cambia. En nuestro caso, esa idea se tradujo en elegir a Random Forest como modelo central, porque captura relaciones no lineales entre permisos, maneja variables binarias sin transformaciones complejas y, al mismo tiempo, ofrece una base natural para analizar la relevancia de las características.

La mejor configuración fue `n_estimators = 200`, `max_depth = 25`, `min_samples_split = 2`, `min_samples_leaf = 1`, `max_features = sqrt` y `class_weight = balanced_subsample`. En validación cruzada estratificada de 10 pliegues alcanzó un *F1-score* promedio cercano a 0.972. En entrenamiento registró *accuracy* de 0.9812, *F1-score* de 0.9812 y *ROC-AUC* de 0.9970, mientras que en prueba llegó a *accuracy* de 0.9703, *F1-score* de 0.9704 y *ROC-AUC* de 0.9942. Estos resultados lo ubican como el mejor entre los modelos evaluados, no solo porque obtuvo el mayor *ROC-AUC*, sino también porque mantuvo el mejor equilibrio general entre capacidad discriminativa, precisión, sensibilidad y estabilidad entre entrenamiento y prueba.

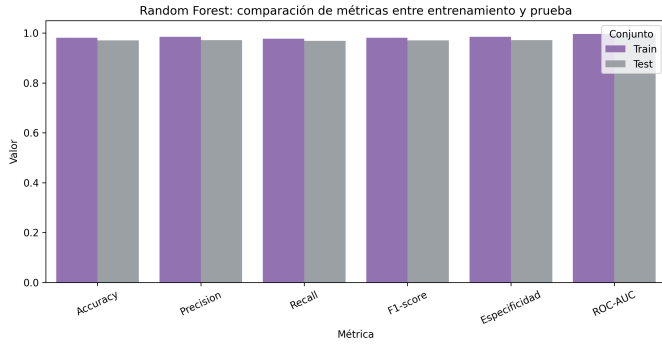


Figura 9: Comparación entre métricas de entrenamiento y prueba para Random Forest.

La Figura 9 muestra que Random Forest conserva métricas muy altas en ambos conjuntos. Aunque existe una brecha moderada entre entrenamiento y prueba, el rendimiento final sobre datos no vistos sigue siendo el más sólido del estudio. En otras palabras, el modelo no solo aprende muy bien el patrón de entrenamiento, sino que también conserva ese comportamiento cuando enfrenta aplicaciones no observadas, lo que es especialmente valioso en un escenario de detección de malware.

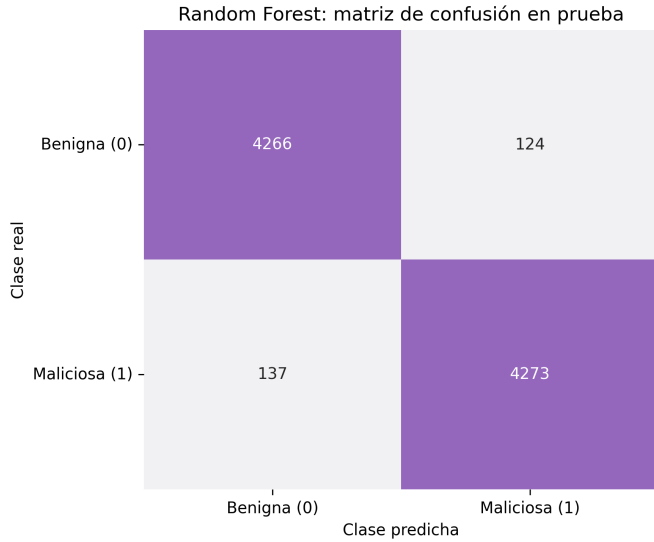


Figura 10: Matriz de confusión de Random Forest sobre el conjunto de prueba.

**IV-B6. Comparación global:** El Cuadro III resume las métricas finales en el conjunto de prueba, mientras que el Cuadro IV resume la relación entre entrenamiento y prueba a partir del *F1-score*. En conjunto, los resultados muestran que Random Forest fue el mejor entre los modelos evaluados. La razón principal es que logró combinar el mayor *ROC-AUC* con un *F1-score* también muy alto y una brecha de generalización controlada. Esto indica que separa mejor las dos clases a lo largo de distintos umbrales y que, además, conserva un balance favorable entre falsos positivos y falsos negativos.

La red neuronal multicapa ocupó el segundo mejor lugar. Su principal fortaleza fue la *precision* más alta del grupo,

lo que significa que, cuando predice malware, se equivoca menos que los demás modelos. Además, alcanzó métricas globales muy cercanas a las de Random Forest. Sin embargo, quedó por debajo de este último porque su *recall* y su *ROC-AUC* fueron ligeramente menores, y porque la brecha entre entrenamiento y prueba fue un poco más evidente.

En contraste, la regresión logística y la SVM lineal ofrecieron los comportamientos más estables entre entrenamiento y prueba. Esa estabilidad es valiosa, pero sus métricas finales fueron inferiores a las de Random Forest y la red neuronal, lo que sugiere que los límites lineales no capturan toda la complejidad de las interacciones entre permisos. KNN, por su parte, mostró un buen nivel de acierto, pero una brecha de generalización más marcada, lo que sugiere mayor sensibilidad a la estructura local del conjunto de entrenamiento y menor robustez frente a datos no vistos.

Cuadro III: Comparación final de métricas en el conjunto de prueba.

| Modelo              | Accuracy | Precision | Recall | F1     | ROC-AUC |
|---------------------|----------|-----------|--------|--------|---------|
| Regresión logística | 0.9590   | 0.9548    | 0.9637 | 0.9593 | 0.9888  |
| SVM lineal          | 0.9591   | 0.9540    | 0.9649 | 0.9594 | 0.9885  |
| KNN                 | 0.9644   | 0.9650    | 0.9639 | 0.9645 | 0.9854  |
| Red neuronal        | 0.9694   | 0.9796    | 0.9590 | 0.9692 | 0.9924  |
| Random Forest       | 0.9703   | 0.9718    | 0.9689 | 0.9704 | 0.9942  |

Cuadro IV: Comparación entre entrenamiento y prueba usando *F1-score*.

| Modelo              | F1 Train | F1 Test | Brecha |
|---------------------|----------|---------|--------|
| Regresión logística | 0.9613   | 0.9593  | 0.0020 |
| SVM lineal          | 0.9615   | 0.9594  | 0.0021 |
| KNN                 | 0.9808   | 0.9645  | 0.0163 |
| Red neuronal        | 0.9805   | 0.9692  | 0.0113 |
| Random Forest       | 0.9812   | 0.9704  | 0.0108 |

Las tablas anteriores permiten comparar de forma directa la generalización de los modelos. En síntesis, Random Forest destacó por su equilibrio entre interpretabilidad, capacidad discriminativa y desempeño final; la red neuronal mostró una capacidad predictiva muy alta, aunque con una generalización ligeramente menos uniforme; y los demás modelos funcionaron como referencias útiles para evidenciar que, en este problema, los enfoques no lineales ofrecen una ventaja clara cuando se busca detectar malware Android a partir de permisos.

## V. REDUCCIÓN DE DIMENSIÓN

### V-A. Análisis individual de variables

Con el objetivo de reducir la complejidad computacional y mejorar la capacidad de generalización de los modelos seleccionados (Random Forest y red neuronal multicapa), se realizó un análisis individual sobre las 86 variables de permisos Android del conjunto de entrenamiento. El análisis combinó tres criterios complementarios: (i) umbral de varianza, (ii) capacidad discriminativa medida mediante información mutua y correlación lineal, y (iii) detección de redundancia por multicolinealidad.

En primer lugar, se aplicó un filtro de varianza con umbral 0.01 para identificar características casi constantes. En segundo lugar, se calculó información mutua (MI) entre

cada variable y la etiqueta objetivo, junto con la correlación de Pearson en valor absoluto. Se marcaron como candidatas a eliminación las variables con baja señal en ambas medidas ( $MI < 0,005$  y  $|r| < 0,05$ ). Finalmente, se inspeccionó la matriz de correlación entre predictores para detectar colinealidad alta ( $|r| > 0,85$ ), con el fin de evitar duplicidad de información.

El conjunto de entrenamiento usado en esta etapa contiene 20532 muestras y 86 variables, con distribución balanceada entre clases (49.88 % benigna y 50.12 % malware). Los resultados confirman que una parte importante del espacio original contiene información poco útil o redundante para la tarea de clasificación.

Por ejemplo, algunos permisos relacionados con sincronización o personalización de fabricantes presentaron muy baja variabilidad o alta redundancia, lo que indica que aparecen de manera casi idéntica tanto en aplicaciones benignas como maliciosas. Este comportamiento limita su aporte discriminativo durante el entrenamiento.

Cuadro V: Resumen del análisis individual de variables (sección 5.1).

| Criterio de filtrado                                          | N. de variables |
|---------------------------------------------------------------|-----------------|
| Varianza casi nula ( $< 0,01$ )                               | 16              |
| Baja capacidad discriminativa ( $MI < 0,005$ y $ r  < 0,05$ ) | 15              |
| Alta redundancia ( $ r  > 0,85$ )                             | 17              |
| Total candidatas a eliminación (unión de criterios)           | 42              |
| Reducción potencial sobre 86 variables                        | 48.84 %         |

Entre las variables con mayor aporte discriminativo se destacaron permisos como `android.permission.READ_PHONE_STATE`, `com.google.android.c2dm.permission.RECEIVE` y `android.permission.RECEIVE_BOOT_COMPLETED`. En contraste, permisos con MI cercana a cero (por ejemplo, `android.permission.WAKE_LOCK`, `android.permission.FLASHLIGHT` y `com.android.vending.CHECK_LICENSE`) aportaron señal limitada para distinguir entre clases.

En síntesis, este diagnóstico sugiere que una parte importante del espacio original puede simplificarse sin afectar de manera significativa la representación esencial del problema. En particular, la detección de 42 variables candidatas a eliminación evidencia la existencia de redundancia estructural y características con bajo valor informativo dentro del dataset. Este resultado respalda la aplicación posterior de técnicas de extracción de características, buscando mantener el desempeño predictivo mientras se reduce la dimensionalidad del problema.

#### V-B. Extracción de características lineal

Para la extracción lineal de características se aplicó el método de Análisis de Componentes Principales (PCA) sobre las variables estandarizadas del conjunto de entrenamiento. El criterio de selección consistió en conservar el número mínimo de componentes capaces de preservar al menos el 95 % de la varianza explicada acumulada, buscando mantener la mayor parte de la información relevante del

dataset con una representación más compacta. Con este criterio se seleccionaron 56 componentes principales a partir de 86 variables originales, lo que representa una reducción dimensional de 34.88 %. Luego se reentrenaron los dos mejores modelos de la sección 4 (Random Forest y red neuronal multicapa) usando el espacio PCA y se evaluaron en prueba.

Los resultados muestran que PCA mantiene un desempeño competitivo incluso después de reducir significativamente la dimensionalidad. En Random Forest se observa una disminución moderada del rendimiento respecto al espacio original ( $F1: 0.9704 \rightarrow 0.9663$ ), mientras que en la red neuronal multicapa la variación es prácticamente mínima ( $F1: 0.9692 \rightarrow 0.9691$ ). Esto indica que la representación lineal preserva gran parte de la información discriminativa necesaria para la clasificación de aplicaciones Android.

Cuadro VI: Comparación en prueba: modelos originales vs. modelos con PCA (56 componentes).

| Modelo                            | Variables | Reducción | Accuracy | Precision | Recall | F1     | ROC-AUC |
|-----------------------------------|-----------|-----------|----------|-----------|--------|--------|---------|
| Random Forest (original)          | 86        | 0.00 %    | 0.9703   | 0.9718    | 0.9689 | 0.9704 | 0.9942  |
| Random Forest + PCA               | 56        | 34.88 %   | 0.9665   | 0.9746    | 0.9580 | 0.9663 | 0.9911  |
| Red neuronal multicapa (original) | 86        | 0.00 %    | 0.9694   | 0.9796    | 0.9590 | 0.9692 | 0.9924  |
| Red neuronal multicapa + PCA      | 56        | 34.88 %   | 0.9692   | 0.9745    | 0.9637 | 0.9691 | 0.9922  |

En términos prácticos, PCA permitió reducir la dimensionalidad del problema en un 34.88 % con una penalización mínima en desempeño predictivo. Esto demuestra que la información relevante del conjunto de datos puede representarse mediante un espacio más compacto sin comprometer sustancialmente la capacidad de clasificación. Por tanto, la extracción lineal mediante PCA constituye una alternativa eficiente cuando se busca disminuir complejidad computacional manteniendo métricas elevadas de desempeño.

#### V-C. Extracción de características no lineal

Para la reducción no lineal se utilizó UMAP sobre el conjunto de entrenamiento, aplicando estandarización previa de las variables para estabilizar el cálculo de vecindad en el espacio de permisos. La selección del número de componentes se realizó evaluando  $k \in \{2, 4, 6, 8, 10, 12\}$  sobre un subconjunto estratificado del entrenamiento. El criterio fue maximizar el promedio de  $F1$ -score en validación entre los dos mejores modelos de la sección 4 (Random Forest y red neuronal multicapa), con una regla de desempate hacia menor complejidad cuando el desempeño fuera equivalente. Con este procedimiento se seleccionó  $k = 10$ , que corresponde a una reducción del 88.37 % frente a las 86 variables originales.

En validación, los mejores promedios de  $F1$ -score se observaron en  $k = 6$  y  $k = 10$  (ambos con 0.8908, redondeado a 4 decimales), y se eligió  $k = 10$  por presentar el mejor compromiso entre ambos clasificadores en la corrida final del notebook. Posteriormente, con ese valor de  $k$ , se reentrenaron ambos modelos sobre todo el conjunto de entrenamiento y se evaluaron en prueba.



Cuadro VII: Resultados de UMAP sobre los dos mejores modelos de la sección 4.

| Modelo                 | $k$ | Reducción | Accuracy | Precisión | Recall | F1     | Especificidad | ROC-AUC |
|------------------------|-----|-----------|----------|-----------|--------|--------|---------------|---------|
| Random Forest          | 10  | 88.37 %   | 0.9160   | 0.9150    | 0.9197 | 0.9173 | 0.9141        | 0.9588  |
| Red neuronal multicapa | 10  | 88.37 %   | 0.9136   | 0.9270    | 0.8984 | 0.9125 | 0.9289        | 0.9603  |

Al comparar con la sección 4, la caída en prueba es clara: Random Forest baja de  $F1=0.9704$  a  $0.9173$ , y la red neuronal multicapa de  $F1=0.9692$  a  $0.9125$ . En otras palabras, UMAP sí logra una compresión fuerte del espacio de entrada, pero en este problema eso tiene un costo importante: los permisos binarios concentran señales muy concretas sobre el comportamiento de la aplicación, y al proyectarlos a pocos componentes parte de esa información se diluye. En un escenario de detección de malware esto es delicado, porque esa pérdida de detalle puede traducirse en más falsos positivos o falsos negativos. Por eso, para este caso, el espacio original termina siendo más útil para separar las clases. UMAP sigue siendo una alternativa válida cuando se busca reducir dimensionalidad y costo de cómputo, pero aquí no fue la opción que mejor conservó el poder predictivo. A diferencia de PCA, la reducción no lineal mediante UMAP produjo una disminución más pronunciada en las métricas de prueba. Este comportamiento es consistente con el análisis individual de variables realizado previamente: aunque existía redundancia en el espacio original, parte de la capacidad discriminativa parecía depender de relaciones distribuidas globalmente entre múltiples características. Debido a que UMAP prioriza la preservación de vecindades locales y la estructura topológica del espacio, parte de la información útil para la generalización pudo verse comprimida o distorsionada durante la proyección. En consecuencia, los resultados sugieren que, para este problema de clasificación tabular basado en permisos Android, una reducción lineal controlada mediante PCA preserva mejor la información relevante para clasificación que una reducción no lineal agresiva como UMAP.

## VI. DISCUSIÓN Y CONCLUSIONES

Los resultados obtenidos a lo largo del proyecto muestran que los modelos supervisados basados en permisos Android constituyen una estrategia efectiva para la detección de malware. Entre los modelos evaluados, Random Forest y la red neuronal multicapa presentaron el mejor desempeño general, alcanzando métricas cercanas al 97 % en Accuracy y F1-score sobre el conjunto de prueba.

El análisis individual de variables permitió identificar redundancia estructural y características con bajo valor discriminativo dentro del espacio original de 86 variables. En particular, se encontró que aproximadamente el 48.84 % de las características eran candidatas potenciales a eliminación debido a baja varianza, escasa relación con la variable objetivo o alta correlación entre predictores. Este resultado evidencia la importancia de aplicar técnicas de selección y extracción de características en problemas de seguridad móvil con espacios dimensionales amplios.

Posteriormente, la aplicación de PCA demostró que es posible reducir la dimensionalidad del problema conservando la mayor parte de la capacidad predictiva. Con

56 componentes principales se preservó el 95 % de la varianza explicada acumulada y se mantuvieron métricas de desempeño muy cercanas a las obtenidas en el espacio original. Esto confirma que una representación lineal compacta puede simplificar el modelo sin introducir pérdidas significativas en clasificación.

En comparación con los trabajos descritos en la sección 3, los resultados obtenidos son consistentes con lo reportado en la literatura reciente. En particular, el desempeño alcanzado por Random Forest coincide con los hallazgos de Mathur et al. [2], quienes reportan precisiones cercanas al 97 % utilizando el dataset NATICUSdroid. Asimismo, los resultados respaldan las conclusiones de Liu et al. [4], donde modelos tradicionales como Random Forest logran desempeños comparables e incluso superiores a enfoques más complejos de aprendizaje profundo. Finalmente, el análisis de reducción de dimensionalidad también se relaciona con lo planteado por Feng y Zhang [1], quienes destacan la relevancia de optimizar espacios de características en entornos dinámicos y de gran escala.

Como conclusión general, el proyecto demuestra que la combinación de ingeniería de características, reducción de dimensionalidad y modelos supervisados permite construir sistemas de detección de malware efectivos, interpretables y computacionalmente eficientes. Además, los resultados obtenidos sugieren que modelos clásicos bien ajustados continúan siendo altamente competitivos frente a alternativas más complejas en tareas de clasificación de malware Android.

## REFERENCIAS

- [1] Z. Feng and X. Zhang, "Supervised incremental feature selection using regularization vector for dynamic multi-scale interval valued datasets," *Pattern Recognition*, vol. 170, p. 111985, 2026.
- [2] A. Mathur et al., "NATICUSdroid: A malware detection framework for Android," *Journal of Information Security and Applications*, vol. 58, p. 102696, 2021.
- [3] B. Sanz et al., "Permission usage to detect malware in Android," *International Joint Conference*, pp. 121–130, 2013.
- [4] G. Liu, D. Caragea, X. Ou, and S. Roy, "Benchmarking Android Malware Detection: Traditional vs. Deep Learning Models," arXiv:2502.15041, 2025. [Online]. Available: <https://arxiv.org/abs/2502.15041>
- [5] A. Alhogail and R. A. Alharbi, "Effective ML-Based Android Malware Detection and Categorization," *Electronics*, vol. 14, no. 8, p. 1486, 2025, doi: 10.3390/electronics14081486.